

15. Nearby Places / Proximity Service (Yelp)

Interviewer: Let's design Yelp's "nearby" search. A user opens the app, we know their lat/lng, and they ask for **coffee shops within 2 km, ranked by rating**. Design the backend that answers that query at scale.

Candidate: Good one — and I want to flag up front how this differs from a ride-hailing "find nearby drivers" problem: here the things being searched are **places that basically never move**. A café doesn't relocate mid-query. That changes consistency, caching, and index-rebuild strategy versus matching a moving driver to a moving rider. Let me clarify first.

1. Clarifying the requirements

Candidate: A few questions before I design: 1. Is the core query "radius search" (within X km), or also arbitrary map-viewport boxes/polygons? 2. How fresh does a *new* listing or an *updated* rating need to be — seconds, minutes? 3. What's the read:write ratio? I'm guessing searches vastly outnumber business edits. 4. Ranking — pure distance, or distance blended with rating/popularity? 5. Global product — dense cities (Manhattan, Tokyo) and sparse rural areas both need to work?

Interviewer: Radius search is the main case; assume viewport/polygon is a stretch goal. A new business or an hours change can take a few minutes to show up — that's fine. Reads outnumber writes by something like 10,000:1. Ranking blends distance and rating. And yes — must work equally well for a dense downtown core and a sparse suburb, without falling over in either.

Candidate: Then here's what I'm building:

Functional - `search(lat, lng, radiusKm, category?)` → `[{placeId, name, distanceKm, rating}, ...]`, sorted by a blend of distance and rating. - `getPlace(placeId)` → full details (hours, photos, review summary). - `createPlace` / `updatePlace` — rare, business-owner-driven.

Non-functional - **Read-heavy, write-light** (~10,000:1) — this shapes the entire architecture toward caching and read-replication, not write throughput. - **Low latency**: interactive map UI, want search p99 under ~150–200 ms. - **Eventual freshness is fine** for place data — no one notices a 2-minute-old hours update. - **Uneven density**: must handle a Manhattan block with 500 restaurants and a rural county with 3, using the *same* index without wasting memory or missing results in either. - Scale: tens of millions of places, tens of thousands of search QPS at peak, concentrated in hot cities at lunchtime.

2. Estimation — why this is an index-shape problem, not a volume problem

Candidate: Let me size it, because the numbers tell you where the *real* difficulty is.

50M places worldwide. Each place ~1 KB (name, coords, category, hours, rating).
Storage: $50M * 1 \text{ KB} \approx 50 \text{ GB}$. <-- trivial. Fits in RAM on a beefy box, even.

Place writes: ~10k edits/day ≈ 0.1 writes/sec. Utterly negligible.

Search reads: 20,000 QPS average, bursty to 2-3x at city lunch rush -> ~50-60k QPS peak, NOT uniform -- concentrated over dense downtown cores (a "hot cell" problem, like a hot-key problem but in 2D space).

=> 50 GB of data and <1 write/sec is NOT a hard problem for a database on its own.
The hard problem is the QUERY SHAPE: "give me every row within X km of (lat,lng)."
lat and lng are two independent dimensions. A B-tree index sorts ONE dimension at a time -- indexing lat alone still leaves you scanning a long thin strip of the globe and filtering by lng in memory. That doesn't get better with more RAM or replicas; it needs an index that's actually built for 2D locality.

Candidate: So unlike the last few problems I've done — which were about surviving a write spike — this one is about **picking the right geospatial index**, then layering read-scaling and caching on top, because the write side is a rounding error.

3. The naive V1 (and why it doesn't scale)

Candidate: Let me state the "obviously wrong at scale" version, to set the baseline.

```
SELECT id, name, lat, lng
FROM places
WHERE lat BETWEEN :lat-delta AND :lat+delta
      AND lng BETWEEN :lng-delta AND :lng+delta
-- then in application code: haversine_distance() on every row, filter by radius
```

Why it dies at scale: a B-tree on `lat` (or a composite `(lat, lng)` index) still has to scan a *rectangular slab* of the whole planet for the `lat` predicate before it can even apply the `lng` filter — in a dense city that slab contains rows from other continents at the same latitude band. At 50M rows and 50k+ QPS, that's a full scan or a wildly imprecise bounding-box scan, then an $O(N)$ haversine pass in app code. Fine for 10K rows; hopeless at real scale. **We need an index where "near in the real world" means "near in the index," in both dimensions.**

4. The centerpiece — geospatial indexing

Interviewer: Right, so how do you actually index two-dimensional space?

Candidate: There are three real options, and the choice is genuinely interesting because of the density requirement you gave me.

Option A — Geohash (string-encode the 2D point)

Interleave the bits of lat and lng into one base32 string. Every extra character narrows the covered box by roughly 5x, and — crucially — **places sharing a prefix are physically close**. That turns a 2D problem into a 1D string-prefix problem, which a normal B-tree (or Postgres/ Elasticsearch string index) already knows how to do fast.

Geohash grid over San Francisco (each cell = one 5-char prefix, uniform size):

```
+-----+-----+-----+
| 9q8yv | 9q8yy | 9q8yz |
|       | * You |       |
+-----+-----+-----+
| 9q8yt | 9q8yw | 9q8yx |
+-----+-----+-----+
```

Query "coffee near me": compute my own cell's prefix (9q8yy), then look up places whose geohash column starts with that prefix -- a plain indexed range/prefix scan, no custom tree structure needed.

Option B — Quadtree (recursively subdivide space by density)

Start with one node covering the whole map; split a node into 4 quadrants once it holds more than, say, 100 places. Repeat recursively.

```
      world
     /  |  |  \
    NW  NE SW  SE
   /|\
  (keeps splitting only where it's crowded)
```

Downtown Manhattan block: split down to tiny cells (each holds ~100 places)
Rural Montana county: stays as ONE big leaf cell (holds ~100 places too)

Option C — Google S2 / Uber H3 (hierarchical cells on a sphere)

Same prefix-locality idea, but on a sphere instead of a flat lat/lng grid, so it avoids the distortion a geohash grid has near the poles/antimeridian. What Google/Uber run in production — worth naming for depth, but I'll reason through geohash vs. quadtree since those are what's implementable and explainable in an interview.

The tradeoff that matters here — geohash vs. quadtree:

	Geohash	Quadtree
Cell size	Fixed per prefix length, uniform everywhere	Adaptive — small cells where dense, huge cells where sparse
Dense city (many places/km ²)	Same-size cell as everywhere else → cell holds way more places than a sparse cell → imbalanced shards , that cell's queries get expensive	Splits automatically until each leaf has a bounded count → balanced query cost regardless of density
Sparse rural area	Wastes tiny cells with near-zero places, or you use very few characters and the cell is enormous (imprecise)	One shallow leaf covers it cheaply — no wasted structure
Implementation	Trivial: it's just a string prefix on a column — reuse a normal B-tree/Elasticsearch index, no custom code	Custom tree structure you build and maintain; rebalancing on insert/delete is real code
Ecosystem	Redis GEOADD / GEORADIUS , Elasticsearch <code>geo_point</code> do this natively	You're building and operating it yourself (or using S2/H3 libraries)

Candidate's call: geohash, backed by Elasticsearch's native `geo_point` / `geo_distance` query, is my default — it's an off-the-shelf string-prefix index, and Elasticsearch already handles sharding, replication, and the neighbor-cell lookup for me. **I'd reach for a quadtree (or S2/H3) specifically because density varies wildly** — a fixed-size geohash cell in Manhattan holds hundreds of restaurants while the same-size cell in rural Montana holds one, so a geohash-sharded-by-prefix system can develop hot, oversized shards in cities. A quadtree's whole design point is adapting cell size to density, so query cost stays roughly even in either place. If I were building the index myself rather than leaning on Elasticsearch, I'd pick quadtree for exactly that reason.

The boundary problem (every geo-index has this — must address it)

```

Cell A      | Cell B
            |
            |
You * | * Actual nearest coffee shop
      | (just across the boundary line,
      | closer to you than anything in
      | your own cell A)

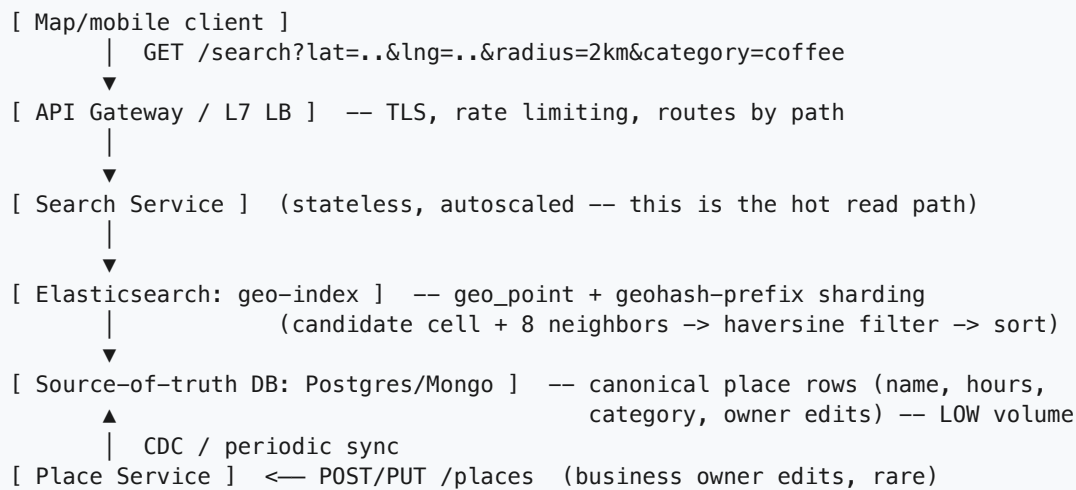
```

If you only query your own cell, you MISS it.
Fix: query your cell + all 8 neighboring cells (a 3x3 block around you, or the equivalent neighbor-walk in a quadtree), collect all candidates, THEN compute true haversine distance and cut at the requested radius.

This neighbor-expansion step is mandatory regardless of geohash vs. quadtree — it's inherent to any grid that carves space into discrete cells.

5. V1 architecture

Candidate: Putting the index into a full picture:



Two stores, two jobs, on purpose: - **Source of truth (Postgres/Mongo):** owns the canonical row. Needs real consistency for owner edits, joins with reviews/photos, backups. Basically never under load. - **Geo-index (Elasticsearch):** exists *purely* to answer "which IDs are near (lat,lng)" fast. It's a **derived, rebuildable** structure — if we lost it entirely, we could rebuild it from the source of truth. That's exactly why it's safe for it to lag by a few seconds.

Why not just add a `geo_point` column to the source-of-truth SQL DB and skip Elasticsearch?

Postgres's `PostGIS / earthdistance` could work for a v0, but Elasticsearch already gives us sharding, replication, and neighbor-aware `geo_distance` queries as a stock feature — building that shard/rebalance/neighbor logic by hand on vanilla Postgres reinvents what a search engine already does well, especially at tens of thousands of QPS.

6. Scaling reads and going multi-region (now 50M places, 50k QPS)

Interviewer: Lunch rush hits Manhattan. Everyone within a half-mile is hitting search at once. Walk me through what happens to your Elasticsearch cluster — and then tell me how this looks once you're global (US, Europe, Asia).

Candidate: This is the hot-cell problem — the 2D equivalent of a hot key. A handful of geohash cells (Midtown, Financial District) get a wildly disproportionate share of the 50k QPS, while most of the planet gets near-zero traffic.

- **Shard the geo-index by region/geohash prefix**, and give each shard **multiple replicas** — Elasticsearch does this natively, so load spreads across replica nodes, not one primary.
- **Cache popular-cell results** (next section) so the *majority* of hot-city queries never even reach Elasticsearch.
- **Read replicas of the source-of-truth DB** for owner-dashboard reads — separate, low-volume concern, kept off the search path.
- Because writes are ~0.1/sec, write-scaling isn't a concern at all — the opposite of a burger-giveaway/flash-sale problem where writes are the fire.

For multi-region, geo-search has an unusually clean sharding key: **the query's own location**, not the user's network location or account region.

[GeoDNS / query-coordinate routing]

- └─ Americas index shard(s) -- places physically in the Americas
- └─ EU index shard(s) -- places physically in Europe
- └─ APAC index shard(s) -- places physically in Asia-Pacific

A search for "coffee near me" while physically in Tokyo NEVER needs data about San Francisco -- routed to the APAC shard regardless of which network/CDN pop received the request.

Unlike a social graph or message feed, where the partition key is fuzzy (which region "owns" a friendship?), geography *is* the partition key here — it falls straight out of the problem.

7. Latency and caching — the heaviest levers in this design

Interviewer: Give me a latency budget for a search call, and walk me through exactly what you cache and how you keep it from going stale.

Candidate: Latency budget first:

Client -> Edge/CDN	~5-20 ms	(geo-routing to nearest PoP)
Edge -> API Gateway -> Search svc	~5-10 ms	
Cache check (Redis, cache hit)	~1-2 ms	<- the overwhelming majority of requests
[cache MISS only] Elasticsearch:		
geohash prefix + 8 neighbor	~15-40 ms	
cells, haversine filter, sort		
Response serialize + return	~2-5 ms	

Cache HIT total:	~15-40 ms	(most traffic, especially popular downtown cells)
Cache MISS total:	~40-90 ms	(rare cell, still well within the ~150-200ms budget)

The single biggest lever is **cache hit rate** — a hit skips Elasticsearch entirely, turning "tens of thousands of QPS against the search cluster" into "tens of thousands of QPS against Redis, with the cluster only seeing the long tail." Because places barely change, this system tolerates aggressive caching better than almost anything else I've designed:

```

SEARCH read path (cache-aside):
  GET /search?lat=37.7701&lng=-122.4110&radius=2km&category=coffee
  |
  |— ROUND (lat,lng) to the enclosing geohash cell <-- critical step
  |   (dedupes near-identical GPS coords into the same cache key;
  |   without this every user's slightly different GPS reading
  |   is its own cache miss, and caching barely helps at all)
  |— key = "search:9q8yy:2km:coffee"
  |— Redis GET
  |   HIT → return cached place-ID list (+ CDN edge cache for the
  |       most popular tourist/downtown cells)
  |   MISS → query Elasticsearch (cell + 8 neighbors, haversine
  |       filter, blend distance+rating, sort)
  |       → write-through to Redis, SHORT TTL (minutes -- a new
  |       coffee shop should surface reasonably soon)
  |
PLACE DETAILS (getPlace):
  cache-aside on place_id, LONGER TTL (hours) -- hours/photos change
  rarely; a specific edit just invalidates that one key directly.

```

Why round the coordinates instead of caching by exact lat/lng? Two users standing 3 meters apart have different raw GPS floats but want the same result set. Rounding to the geohash cell turns "everyone's slightly unique query" into "a small, highly-reused set of cache keys" — this single trick is most of why caching works so well here.

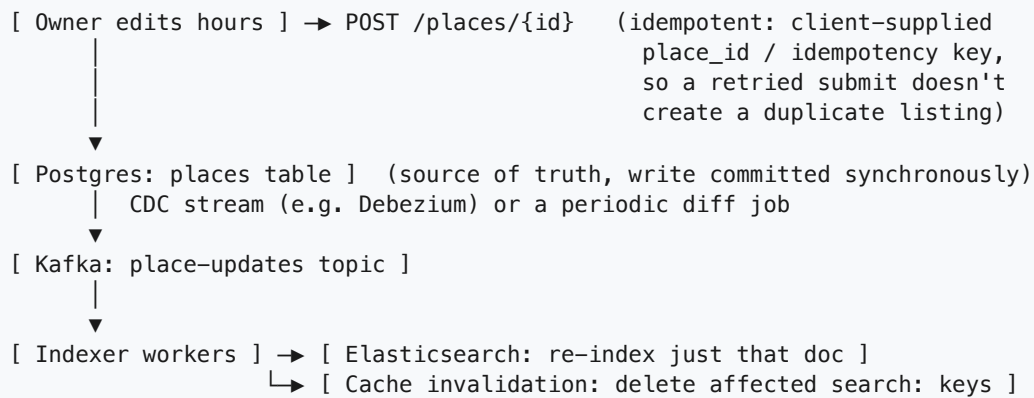
8. Consistency — what's strong, what's eventual, and why

Interviewer: A restaurant owner updates their hours. How fast does that need to propagate, and what's your consistency story end to end?

Candidate: - **Strong / read-your-own-write:** the owner's own dashboard, reading straight from the source-of-truth DB, should show their edit immediately — that's just a normal DB read after a normal DB write, no tricks needed. - **Eventual, by design, everywhere else:** the geo-index (Elasticsearch) and the search cache are both *derived* views. An edit propagates to them via CDC/periodic sync, and a searcher a few minutes later sees the update. Nobody notices or cares — this is the same tolerance for staleness as an immutable URL-shortener mapping, just because writes are so rare rather than because the data can't change at all. - **The one thing I would NOT make eventual:** don't let a place get "orphaned" — deleted from the source of truth but still findable in a stale index for days. Set a max staleness bound (e.g., full incremental resync at least every few minutes) so "closed" places age out in a bounded window, not indefinitely.

9. Async — what moves off the request path

Candidate: Nothing about *search* needs to be async — it's a pure read, synchronous, done. The async work is entirely on the **write side**, which is convenient because writes are rare and never latency-sensitive:



- The owner's `POST` returns as soon as Postgres commits — they don't wait for Elasticsearch or cache invalidation.
- **CDC into Kafka, not a direct dual-write from the app**, because dual-writing to Postgres and Elasticsearch in the request handler risks one succeeding and the other failing (now they disagree, silently). CDC guarantees the index eventually reflects every committed row, replayable if the indexer crashes.
- **Full index rebuild** happens rarely (schema/algorithm changes) and swaps in atomically — build the new index/segment in the background, flip a pointer/alias, never let a reader see a half-built index.

10. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Geo-index for "nearby" queries	Geohash via Elasticsearch geo_point / geo_distance (quadtree/S2 as the density-adaptive alternative)	String-prefix locality reuses an off-the-shelf B-tree-style index; Elasticsearch handles sharding, replicas, and neighbor-cell logic for free. <i>Not a plain SQL (lat, lng) B-tree</i> — a single-dimension index can't answer a 2D radius query without a huge bounding-box scan. <i>Quadtree instead</i> , if I'm rolling my own index and density varies wildly (dense city vs. rural) — its adaptive cell size keeps query cost even; a fixed geohash grid can develop oversized, hot cells in dense areas.
Source of truth for place records	PostgreSQL (or MongoDB)	Canonical row needs real consistency for owner edits, joins with reviews/photos, easy backup/restore. Volume is trivial (50 GB) — this store is never the bottleneck.
Search/read scaling	Elasticsearch shards + replicas, geo-sharded by region	Search load (tens of thousands of QPS) is the only real scaling pressure in this system; replicas absorb it horizontally, and region-sharding means a Tokyo query never touches the Americas shard.
Cache in front of the index	Redis, keyed by rounded geohash cell + radius + category	Places change rarely, so a short-TTL cache absorbs the overwhelming majority of repeat/popular-area queries and keeps Elasticsearch load far below its

Concern	Choice	Why this, and why not the alternative
		ceiling. <i>Not caching by raw lat/long</i> — every user's slightly different GPS reading would be a unique key and the cache would barely hit.
Write propagation, source-of-truth → index	CDC (e.g. Debezium) → Kafka → indexer workers	Guarantees the index eventually reflects every committed write, replayable on indexer crash. <i>Not a synchronous dual-write</i> from the app to both Postgres and Elasticsearch — that risks the two disagreeing if one write fails and the other doesn't, silently.
Idempotency on writes	Client-supplied <code>place_id</code> / idempotency key on create	Writes are rare but still need this — a retried "add my business" submission on a flaky network must not create a duplicate listing.
CDN	Edge cache for static place assets + hottest search cells	Photos/descriptions are static; the most popular downtown cells' search results can even be edge-cached briefly, shaving the last few ms off the most common queries.

The one-sentence justification you'd say out loud: *"Because places are read 10,000x more than they're written, I push almost all the design effort into the read path: a geohash-backed Elasticsearch index (or a quadtree if I'm building it myself, to stay density-adaptive between Manhattan and rural areas) sharded by region and replicated for the hot-city QPS, a Redis cache keyed by rounded geohash cell that absorbs the bulk of repeat queries, and a source-of-truth SQL store that only has to handle a trickle of owner edits propagated asynchronously via CDC — so staleness after an edit is a deliberate, safe tradeoff, never a correctness problem."*

11. Failure modes & graceful degradation

Failure	What happens	Mitigation
One Elasticsearch shard replica dies	That shard's queries briefly degrade	Other replicas of the shard keep serving; rebuild the lost replica from the surviving ones/source DB
Entire geo-index cluster down	Search would hard-fail	Degrade to cached results only, or a coarse SQL bounding-box scan as a slow fallback — never a blank error page
Source-of-truth DB down	Search is UNAFFECTED — it never reads from there on the hot path	Owner edits queue/fail gracefully; reconcile once DB recovers
Redis cache down	Latency jumps (every query now hits Elasticsearch)	Functional but slower; Elasticsearch alone can still serve at reduced headroom; alert and recover cache ASAP
CDC/Kafka lag	New places / edits take longer than usual to appear in search	Already an eventual-consistency system — a longer delay is a graceful degradation, not an outage

Failure	What happens	Mitigation
Index rebuild mid-flight	Risk of readers seeing a half-built index	Build new index/segment in the background, atomic pointer/alias swap on completion
Hot cell overload (viral location)	One geohash cell's shard gets hammered	Cache absorbs most of it; if it's persistent, split that cell further (quadtree) or add dedicated replicas for that shard

12. Follow-up curveballs

Interviewer: How do you rank results — pure distance, or something else?

Candidate: Distance narrows the *candidate set* (the geo-index's job), but ranking is a separate concern layered on top: blend distance with rating, popularity, and maybe "open now." I'd compute a score like `score = w1*ratingNorm + w2*(1 - distance/radius)` and sort candidates by that after the geo-index returns them — keeping ranking logic out of the index itself means I can tune the ranking formula without touching how geo-lookup works.

Interviewer: What about a user driving, hitting search repeatedly as they move?

Candidate: Debounce client-side (don't fire a new search every 50 meters), and the rounded-geohash-cell cache absorbs most of the repeat queries anyway since a car moving a few hundred meters usually stays in the same cell or its immediate neighbor. This is a much lighter version of the ride-hailing driver-matching problem — there, the *thing being searched for* is also moving and needs constant re-indexing; here, only the *searcher* moves, and the places they're searching stay put. That asymmetry is why this system can lean so hard on caching where ride-hailing can't.

Interviewer: Support arbitrary polygon search — "everything inside this map viewport I drew," not just a radius.

Candidate: Elasticsearch supports `geo_shape / geo_polygon` queries natively, so it's a different query type against the same underlying index — no change to how documents are indexed. If I were doing it with a raw quadtree, I'd walk the tree collecting cells that intersect the polygon's bounding box, then do exact point-in-polygon filtering on that candidate set, same shape as the radius boundary-problem fix.

Interviewer: A category ("coffee") has almost no results in a rural search area. What happens?

Candidate: The neighbor-cell expansion (querying the 8 surrounding cells) already helps, but if it's still sparse, expand the search radius progressively (2km → 5km → 10km) rather than returning an empty list, and say so in the response ("nearest coffee shop is 8km away") so the UI can be honest about it instead of silently failing.

13. Deep-dive: "Why not just run a SQL query with `WHERE haversine(...) < r`?"

Interviewer: Why do I need a geospatial index at all? Every place row has a `lat / lng`. Why can't I just write `SELECT * FROM places WHERE haversine(lat, lng, :myLat, :myLng) < :r` and let the database compute the distance for every row? Put a B-tree on `(lat, lng)` if it's slow.

Candidate: This is the version everyone reaches for first, and it's worth being precise about exactly *why* it falls over, because the failure isn't "SQL is slow" — it's that **`haversine()` is a computed function of two columns, and no ordinary index can help evaluate an inequality on a function's output.**

Why the full scan happens

`haversine(lat, lng, myLat, myLng) < r` is not a predicate on a column value — it's a predicate on the *output of a function call* applied to every row. A B-tree only helps when the database can seek directly to a contiguous range of an indexed column that satisfies the `WHERE` clause. There's no index on "distance from an arbitrary point I'll supply at query time," so Postgres can't seek at all — it must **evaluate the function once per row**, then discard the ones that fail:

50M rows, no way to seek → the DB visits ALL 50M rows, computes trig (haversine uses `sin/cos/atan2` — not cheap) on each one, then throws away all but the handful within `r` km. This is a FULL TABLE SCAN with expensive per-row math, not a fast lookup that happens to be "a bit slow."

At 50k+ search QPS, that's $50k * 50M$ row-evaluations/sec — the database would need to be doing quadrillions of trig calls per second. It doesn't matter how much RAM or how many read replicas you add; the WORK PER QUERY doesn't shrink.

Why a plain `(lat, lng)` B-tree doesn't rescue it

A composite B-tree index on `(lat, lng)` sorts rows by `lat` first, and only *within* equal `lat` values does it sort by `lng`. That's a **1D sort**, not a 2D one — it can accelerate `lat BETWEEN a AND b`, but the moment you add `lng BETWEEN c AND d`, the index can only narrow to the *matching lat range* and then has to check every row in that range's `lng` values one-by-one, because rows with the same latitude are scattered across every longitude on Earth:

B-tree on `lat`, sorted:
... 37.76, 37.76, 37.77, 37.77, 37.77, 37.78 ...
 ^-- all rows at `lat 37.77` could be anywhere from Tokyo to Toronto
 to San Francisco — `lng` isn't sorted within this slice at all.

So "`lat BETWEEN 37.76 AND 37.78`" seeks efficiently... but then every one of those rows (a full latitude BAND around the entire planet) has to be filtered by `lng` and then by real distance, in application code or a second pass. Two independent 1D indexes (or one composite) still can't express "near in 2D" as a single contiguous range.

This is exactly the point from §3: **lat and lng are two independent dimensions, and a B-tree only orders one dimension at a time.** A geohash/quadtree/S2 cell number is deliberately constructed so that "near in 2D space" *does* collapse into "adjacent in a 1D sort order" (a prefix or a cell ID) — that's the entire trick, and it's precisely what a raw `lat/lng` B-tree cannot do no matter how it's composed.

	Naive haversine() scan	Geohash prefix	Quadtree	S2 / H3
Underlying idea	Compute distance for every row, filter	Interleave lat/long bits into a sortable string; prefix = region	Recursively subdivide space into 4 quadrants by density	Project onto a sphere, tile with hierarchical cells (S2: quad-sphere; H3: hexagons)
Query cost at 50M rows	O(N) full scan + trig per row — same cost whether 5 or 5M rows match	O(log N) prefix seek + small candidate set	O(log N) tree descent + small candidate set	O(log N) cell lookup + small candidate set
Reuses a plain B-tree?	Technically yes, but the index can't help evaluate the function	Yes — it's just a string column, off-the-shelf index	No — custom tree structure, or a library (e.g. PostGIS GiST)	No — needs an S2/H3 library; can backstop with a normal index on the cell-ID column
Handles uneven density	N/A — cost is bad everywhere regardless of density	Poor — fixed cell size, dense cells get overloaded	Good — adaptive cell size by construction	Good — resolution can be chosen/varied per query, but base cells are still fixed-size until you pick a level
Distortion near poles / antimeridian	None (exact haversine every time)	Real — flat-grid geohash squares get distorted near poles	N/A (space-partition, not grid-projection)	None — designed to be near-uniform on a sphere
Implementation cost	Trivial to write, catastrophic to run	Low — string column + prefix query	Medium-high — you build/maintain the tree	Medium — adopt a library (Google S2, Uber H3), integrate cell IDs
When it's actually fine	Tiny N (thousands of rows) or a pre-filtered small candidate set	Default choice for most products (this is what we picked in §4)	Density varies by orders of magnitude and you're rolling your own index	Global scale, need sphere accuracy, or want variable resolution per query (Uber's actual use case)

Rule of thumb: If you can't express "nearby" as a contiguous range on an indexed column, the database will scan everything to find out — no amount of hardware fixes an O(N)-per-query algorithm. A geospatial index's only job is to make "near in the real world" collapse into "a short, seekable range in the index." Geohash does that with a string; quadtree does it with a tree; S2/H3 does it with cells on a sphere — pick based on density-uniformity and distortion tolerance, not because one is fashionable.

14. Deep-dive: the boundary problem *and* uneven density, walked through together

Interviewer: Take your geohash design and break it for me. Give me a query that returns the wrong answer, and then tell me what happens when I point it at Manhattan instead of rural Wyoming.

Candidate: These are two distinct failure modes that both fall out of the same root cause — **a grid cell is an arbitrary human-drawn box, and the real world doesn't respect it**. Let me walk through each concretely.

Failure 1 — the boundary problem

A user standing one meter from a cell edge has their query scoped to *their own cell only*. The nearest coffee shop in the real world might be five meters away — but on the *other* side of that line, in a different cell — and gets missed entirely, while we happily return a place that is technically "in the same cell" but 1.9 km away.

Cell 9q8yv (geohash prefix A)		Cell 9q8yy (geohash prefix B)
☺ Roasted Bean (1.9 km away, same cell as you)		* YOU (query point, 40m from the boundary line)
		☺ Corner Café (only 90m away -- the TRUE nearest -- but lives in cell B)

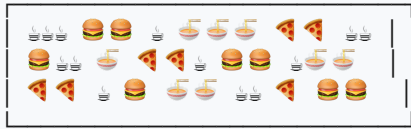
Query scoped to prefix = "9q8yv" only:
SELECT * FROM places WHERE geohash LIKE '9q8yv%'
returns Roasted Bean (1.9km) and MISSES Corner Café (90m) entirely --
not because it's far, but because it's on the wrong side of an
imaginary line that has nothing to do with real distance.

The bug isn't rare — it's *guaranteed* to happen for every query point near a cell edge, and with a fine-grained grid (needed for density in cities), a huge fraction of real queries land near *some* edge.

Failure 2 — uneven density (the same grid, a different symptom)

A fixed-precision geohash cell covers the same physical area everywhere on Earth — say roughly 150m x 150m at 7 characters. That's a reasonable bucket size for rural Wyoming. It is a catastrophically bad bucket size for Midtown Manhattan:

Manhattan (one 7-char geohash cell, ~150m x 150m block):



<-- 400+ places crammed into
ONE cell/shard/cache key.
Every lunch-rush query here
hits the same hot shard.

Rural Wyoming (one 7-char geohash cell, SAME ~150m x 150m):



<-- 0-1 places. This cell (and
thousands like it) wastes
index/shard overhead on
almost nothing.

Same grid, wildly different load. The Manhattan cell becomes a **hot-cell/hot-shard problem** (§6) — its Elasticsearch shard gets hammered — while thousands of rural cells sit nearly idle, wasting sharding/replication overhead on empty space. Fixed-size cells cannot win in both regimes at once: shrink the cell size to fix Manhattan and you multiply the number of near-empty cells everywhere else; grow it to stop wasting cells in Wyoming and Manhattan's cells get even more overloaded.

Ranked fixes

Fix 1 — query the cell + its 8 neighbors (mandatory baseline, fixes the boundary problem)

```
Instead of: WHERE geohash LIKE '9q8yy%'
Do:         WHERE geohash IN ('9q8yv', '9q8yw', '9q8yx',
                              '9q8yy', '9q8yz', '9q8yt',
                              '9q8y6', '9q8y7', '9q8y8') -- self + 8 neighbors
ORDER BY haversine(...) LIMIT k
```

Fetch the query point's cell plus all 8 cells surrounding it (a 3x3 block; the equivalent is a neighbor-walk in a quadtree), collect every candidate from all 9, compute *true* haversine distance on that small candidate set, then filter to the requested radius. This is cheap — 9 cells instead of 1 is still a tiny scan compared to a full table scan — and it's what already made it into §4. **This fixes boundary correctness. It does nothing for density.**

Fix 2 — variable-precision cells / a quadtree that subdivides dense areas (fixes density)

Don't force every cell to the same precision. Two ways to get this: - **Quadtree** (§4, Option B): split a cell into 4 once it holds more than, say, 100 places. Manhattan recurses down to tiny leaves; Wyoming stays one big leaf. Query cost per cell is now bounded everywhere. - **Variable-precision geohash**: store *each place* at whatever geohash length keeps roughly even occupancy — dense areas get longer (more specific) prefixes, sparse areas get shorter ones — and query by trying multiple precisions, or maintain a lookup of "what precision is cell X actually populated at." More fiddly than a quadtree because geohash wasn't designed for variable depth per region; a quadtree gets this for free by construction.

Fix 3 — search-radius-to-precision mapping (an optimization, not a correctness fix)

Pick geohash precision *based on the requested search radius*, not a single fixed length for the whole dataset: a 500m radius search should probe finer (longer-prefix) cells than a 20km radius search, so you're not fetching a huge 3x3 block of coarse cells and then discarding 99% of candidates by distance, or conversely fetching too-fine cells and needing dozens of neighbors to cover the requested radius. This tunes *how many* neighbor cells fix 1 needs to touch, and pairs naturally with fix 2's adaptive structure.

Recommended approach: combine fix 1 and fix 2, and treat fix 3 as tuning on top. Fix 1 (neighbor expansion) is **non-negotiable** — every grid-based geo-index has the boundary problem and must always over-fetch neighbors before trusting a result set, full stop. Fix 2 (adaptive cell size — quadtree, or S2/H3 with per-region resolution) is what actually solves the density imbalance so Manhattan's shard isn't 1000x hotter than Wyoming's; it's a structural choice made when building the index, not a per-query patch. If leaning on Elasticsearch's native geohash (as in §4), get adaptive behavior "for free" from geo-sharding + replicas absorbing the hot cells, plus the cache from §7 doing most of the work — but if rolling a custom index, pick quadtree/S2 specifically *because* density varies by orders of magnitude between a city block and a rural county, exactly as argued in §4.

What to say out loud: *"Every cell-based geo-index has the same two failure modes for the same reason — a grid cell is an arbitrary boundary the real world ignores. Near an edge, the true nearest result can be one meter away in the next cell, so I always expand to the cell plus its 8 neighbors before trusting a result set — that's non-negotiable and cheap. Density is the harder one: a fixed-size cell that's right for rural Wyoming is wildly wrong for Manhattan, so I want a structure — quadtree, or variable-resolution S2/H3 — that subdivides where it's crowded and stays coarse where it's empty, so query cost and shard load stay roughly even everywhere instead of concentrating in a handful of hot cells."*

Summary — the mental model

This problem is **not about surviving a write spike — writes are a rounding error — it's about picking an index shape for two-dimensional locality and then leaning hard on caching**, because places are read thousands of times more than they're written and barely change once created. The winning moves: a **geohash (or density-adaptive quadtree/S2) geo-index** that turns "near in the real world" into "near in the index," always paired with a **neighbor-cell expansion** to avoid missing places just across a boundary line; **region-sharded, replicated Elasticsearch** to absorb hot-city read spikes; a **Redis cache keyed by rounded geohash cell** that turns near-duplicate GPS queries into cache hits; and a **source-of-truth SQL/Mongo store** that only has to handle a trickle of owner edits, propagated to the index **asynchronously via CDC** so staleness after an edit is a deliberate, harmless tradeoff rather than a bug to fix.